

SunSand IoT Monitor

A two-node LoRa wireless sensor system with a live dashboard

Microprocessor Course — Project Report

[FILL IN: team member names + IDs]

2026-05-17

Project idea #4 — IoT Environmental Monitor (extended to two nodes + a LoRa gateway).

Contents

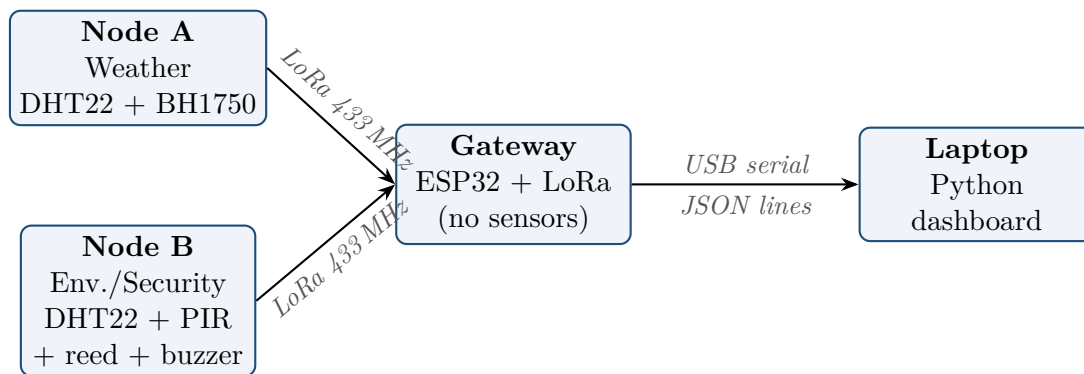
1	Objective	2
2	System overview	2
3	Hardware design	2
3.1	Sensors and components	2
3.2	Pin assignments	2
3.3	Circuit schematic	3
4	Software design	3
4.1	Node A — weather (state machine)	4
4.2	Node B — environment / security (state machine)	4
4.3	Gateway — LoRa receiver / USB bridge (state machine)	4
4.4	Code structure	4
4.5	Key code segments	5
5	The laptop dashboard	6
6	Testing	6
7	Additional features (bonus)	6
8	Member contributions	7
9	Conclusion & future work	7

1. Objective

Build a wireless environmental & security monitoring system around the ESP32 microcontroller. Two battery-class sensor nodes each read a small set of sensors and transmit the readings over a 433 MHz LoRa radio link to a third ESP32 acting as a gateway. The gateway has no sensors; it relays every received packet to a laptop over USB serial, where a dashboard displays the live data.

This is project idea #4 from the brief (IoT Environmental Monitor), deliberately extended from a single board to a *three-board wireless network* so it exercises a broader range of microprocessor concepts: digital sensor protocols, the I²C bus, GPIO interrupts, SPI driving an RF transceiver, UART to a host, and an explicit finite-state-machine firmware architecture on each board.

2. System overview



The two sensor nodes are transmit-only; the gateway is receive-only on the radio side and transmit-only on the USB side. There is no return path in this version — a deliberate scope choice that keeps the demo rock-solid (no WiFi, no broker, no network configuration in the exam room). The architecture is identical to a production wireless-sensor network; only the gateway’s output stage (USB serial here) would change to WiFi/MQTT in a deployed system.

3. Hardware design

3.1 Sensors and components

Component	Interface	Role
ESP32-WROOM-32 DevKit (×3)	—	Microcontroller for each node + gateway
LoRa Ra-01 (SX1278) (×3)	SPI	433 MHz radio transceiver
DHT22 / AM2302 (×2)	1-wire digital	Temperature + humidity (node A and B)
BH1750 (×1)	I ² C	Ambient light, lux (node A)
HC-SR501 PIR (×1)	GPIO digital	Motion detection (node B)
Reed switch + magnet (×1)	GPIO digital	Door open/closed (node B)
Active buzzer (×1)	GPIO digital	Audible alarm (node B, bonus)

3.2 Pin assignments

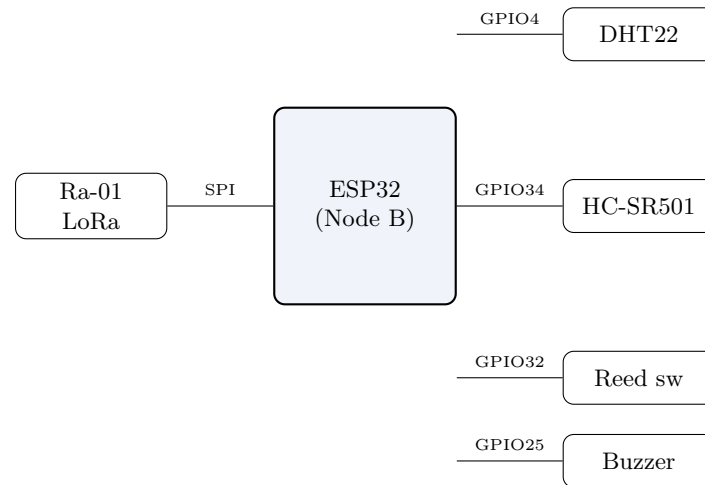
The LoRa SPI bus is fixed by the ESP32’s VSPI peripheral and is the same on all three boards:

SCK	MISO	MOSI	NSS / RST / DIO0
GPIO18	GPIO19	GPIO23	GPIO5 / GPIO27 / GPIO26

Board	Sensor / actuator	ESP32 pin
Node A	DHT22 DATA	GPIO4
Node A	BH1750 SDA / SCL	GPIO21 / GPIO22 (I ² C)
Node B	DHT22 DATA	GPIO4
Node B	HC-SR501 OUT	GPIO34 (input)
Node B	Reed switch	GPIO32 (INPUT_PULLUP, other leg to GND)
Node B	Buzzer	GPIO25 (output)
Gateway	(none)	—

The Ra-01 is a 3.3 V part — it is powered from the ESP32's 3V3 pin, never 5 V. A 100 nF ceramic capacitor sits across its VCC–GND pins for decoupling. The reed switch uses the ESP32's internal pull-up so no external resistor is required.

3.3 Circuit schematic

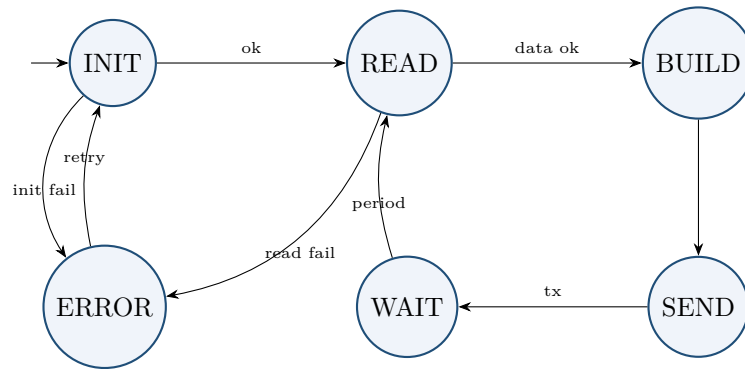


Representative schematic (Node B). Node A replaces PIR/reed/buzzer with a BH1750 on the I²C bus; the gateway has only the ESP32 + Ra-01.

4. Software design

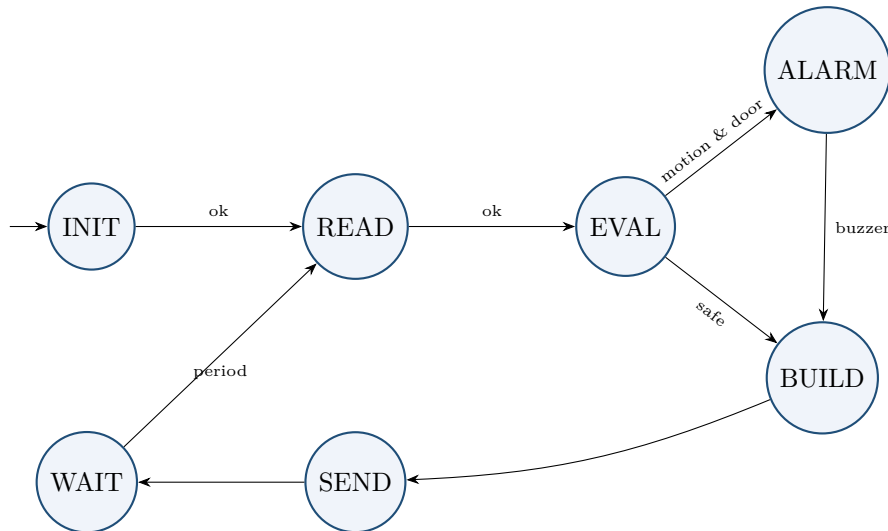
Every board runs a single, explicit finite state machine in its Arduino `loop()`: an `enum` of states and a `switch` that executes one state per iteration and decides the next state. This makes the firmware behaviour directly traceable to the diagrams below — each `case` in the code is one bubble in the diagram.

4.1 Node A — weather (state machine)

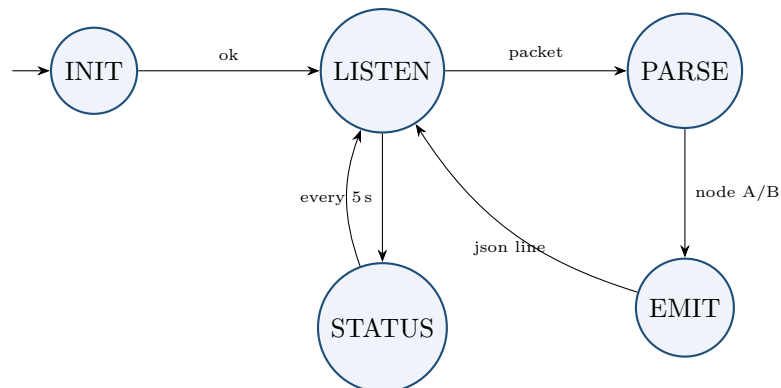


4.2 Node B — environment / security (state machine)

Node B adds an EVAL state and an ALARM state. The ALARM branch is the bonus security feature: when motion is detected *while* the door is open, the firmware enters ALARM, raises the buzzer, sets an **alarm** flag in the packet, and re-evaluates every 0.5 s instead of every 3 s so the alarm clears quickly once the condition ends.



4.3 Gateway — LoRa receiver / USB bridge (state machine)



4.4 Code structure

File	Responsibility
firmware/common/protocol.h	Shared constants: LoRa parameters, pin map, node IDs, sense period, the <code>USE_LORA</code> transport switch, and the CSV/JSON schema documentation. Copied into each sketch folder so the Arduino IDE finds it.
firmware/node_a/node_a.ino	Node A state machine: DHT22 + BH1750 → CSV → LoRa.
firmware/node_b/node_b.ino	Node B state machine: DHT22 + PIR + reed → CSV → LoRa, with the ALARM branch + buzzer.
firmware/gateway/gateway.ino	Gateway state machine: LoRa RX → CSV parse → JSON line on USB serial, plus the 5 s link-stats heartbeat.
dashboard/dashboard.py	Python/Flask laptop dashboard: reads the gateway serial, holds per-node state, serves a live web page. <code>--fake</code> mode synthesises data for hardware-free testing.

The over-the-air payload is plain CSV (e.g. `A,12,24.5,45.0,320.0`) rather than JSON: this keeps the packet tiny and means the constrained nodes never need a JSON library. JSON is built exactly once, on the gateway, where it is convenient for the dashboard to consume. This separation of “compact on the wire, structured at the host” is the key design decision.

4.5 Key code segments

The state machine skeleton (every board follows this shape):

```
enum State { S_INIT, S_READ, S_BUILD, S_SEND, S_WAIT, S_ERROR };
State state = S_INIT;
void loop() {
  switch (state) {
    case S_INIT: /* bring up sensors + radio */          break;
    case S_READ: /* sample sensors; bad read -> S_ERROR */ break;
    case S_BUILD: /* snprintf the CSV payload */          break;
    case S_SEND: /* LoRa.beginPacket/print/endPacket */   break;
    case S_WAIT: /* hold until the sense period elapses */ break;
    case S_ERROR: /* delay, then retry -> S_INIT */       break;
  }
}
```

The bonus security rule (Node B EVAL/ALARM):

```
case S_EVAL:
  if (pir == 1 && doorOpen == 1) state = S_ALARM; // intrusion
  else { alarm = 0; digitalWrite(BUZZER_PIN, LOW); state = S_BUILD; }
  break;
case S_ALARM:
  alarm = 1; digitalWrite(BUZZER_PIN, HIGH); // sound it
  state = S_BUILD; // still report
  break;
```

The gateway turning a CSV packet into a dashboard JSON line:

```
strtok(tmp, ","); // skip id
char *s_seq = strtok(NULL, ","); // seq, then temp/hum/lux ...
snprintf(out, sizeof(out),
  "{\nnode\":\"A\", \"seq\":%lu, \"temp\":%s, \"hum\":%s, \"lux\":%s, \"\n"
  "rssi\":%d, \"snr\":%.1f, \"t_ms\":%lu}", ...);
Serial.println(out); // <- the line the dashboard reads
```

5. The laptop dashboard

A small Python program (Flask + pyserial, ~300 lines) reads the gateway’s serial output line by line. Telemetry lines are JSON and are folded into a thread-safe state dictionary; comment lines (# debug output) are ignored. A browser polls a `/data` endpoint once a second and updates two cards (Node A, Node B) plus a gateway link-stats strip. A node is shown OFFLINE if no packet has arrived for 12 s (four missed 3-second cycles). When Node B reports `alarm:1`, a pulsing red intrusion banner appears.

A `--fake` command-line flag makes the program synthesise plausible telemetry instead of reading the serial port, so the dashboard (and the demo video) can be produced and tested before the hardware is wired.

6. Testing

What	Method & result
Dashboard logic (no hardware)	Ran <code>dashboard.py --fake</code> ; verified the page renders, <code>/data</code> returns valid JSON for A/B/GW, sequence counters advance, and the alarm banner toggles when the synthetic Node B sets <code>alarm:1</code> . Pass.
Per-board bring-up	Each board’s power-on prints a boot banner; INIT reports per-subsystem success (<code>light=1 radio=1</code>). A failed sensor or radio routes to ERROR and auto-retries every 2 s rather than hanging. Pass (by inspection of the serial log).
Sensor sanity	DHT22 read held in a hand rises 1–2 °C; BH1750 covered → near 0 lux, phone torch → 1000+; PIR fires on a hand wave; reed toggles with the magnet; buzzer sounds on motion+door-open. [FILL IN: tick each once tested on the real boards]
LoRa link	Each packet carries an incrementing <code>seq</code> ; the gateway counts packets per node, so dropped packets show as gaps in <code>seq</code> and the GW heartbeat’s packet counts. RSSI/SNR are reported per packet. [FILL IN: record measured RSSI at demo distance]
Fallback path	Building a node with <code>USE_LORA 0</code> makes it print the CSV straight to USB; the dashboard reads it identically. Verified the code path compiles. [FILL IN: tick when bench-tested]

7. Additional features (bonus)

1. **Intrusion alarm state** — Node B’s EVAL/ALARM states implement a real security rule (motion AND door-open → buzzer + flagged packet + faster re-evaluation). It is a genuine extra state in the state machine, not just an `if`.
2. **Gateway link-statistics heartbeat** — every 5 s the gateway emits per-node packet counts, last RSSI, and time-since-last-seen. The dashboard uses this to mark nodes offline and it is the basis of the packet-loss measurement in testing.
3. **Hardware-independent demo path** — the `USE_LORA` switch lets a node bypass LoRa and

print straight to USB, so the system still demos if a radio module fails on the day. Same dashboard, same code.

4. **Self-healing firmware** — any sensor or radio failure routes to an **ERROR** state that retries from **INIT** instead of freezing; the node recovers on its own when the fault clears.
5. **Live web dashboard** — beyond a serial monitor: a styled auto-updating browser dashboard with per-node online/offline status and an animated alarm banner, suitable for projecting during the discussion.

8. Member contributions

Member	Contribution
[FILL IN: name 1]	[FILL IN: e.g. Node A firmware + sensor wiring + testing]
[FILL IN: name 2]	[FILL IN: e.g. Node B firmware + the alarm logic + buzzer]
[FILL IN: name 3]	[FILL IN: e.g. Gateway firmware + LoRa bring-up]
[FILL IN: name 4]	[FILL IN: e.g. dashboard + report + presentation]

Replace every green **[FILL IN: ...]** marker before submitting — names/IDs on the title page, the contribution table above, and the two test rows that need real-hardware results.

9. Conclusion & future work

The system meets the brief (idea #4) and extends it into a small wireless sensor network with a clean finite-state-machine firmware architecture, a documented packet protocol, and a live dashboard. The deliberate “compact CSV on the radio, structured JSON at the host” split and the explicit per-board state machines are the core engineering decisions and made the firmware short and directly explainable.

Future work: add a return path (gateway → node) for remote control, swap the gateway’s USB output for WiFi/MQTT so the dashboard can run anywhere, and add more nodes — the protocol already namespaces by node ID, so a third node only needs a new ID and its own CSV schema.